08 / 17 — Phase 8 Assessment: Testing and Quality

> **Scope.** Lessons 01 through 16 of Phase 8 (test strategy, Vitest, React Testing Library, async updates, custom hooks, mocking and MSW, API interactions, integration testing, Playwright E2E, accessibility testing, component contracts, visual regression, TypeScript correctness, linting and static analysis, CI checks, avoiding brittle tests), with Phases 0–7 knowledge assumed. **Format.** Five parts: (A) conceptual explanation, (B) code prediction, (C) debugging challenges, (D) implementation exercises, (E) architecture and reasoning. A detailed answer key, scoring rubric, and promotion criteria follow. **Time.** 5–6 hours, closed notes for Parts A–C and E. Part D may be done in an editor with Vitest, Testing Library, MSW, TypeScript and Playwright installed (no AI assistance, no copying from lessons). **Rules.** Write answers *before* reading the key. For every prediction, state the outcome and a sentence of reasoning. For every debugging challenge, name the root cause, the fix, and how you would verify the fix. Answers that correctly say "verify against current docs" for volatile tool APIs are acceptable and rewarded; confidently stating wrong API details is not. Version baseline: React 19.x, TypeScript 5.x, Vitest, Testing Library (`user-event` v14), MSW 2.x, Playwright (see ROADMAP §0). The Part B predictions (B1, B2, B3, B5) and the B6 compiler behaviour were executed on 2026-10-22 (React 19.3.1, RTL 16.3.3, user-event 14.6.8, Vitest 3.2.7, jsdom 30, TypeScript 5.9.3) and the stated outcomes held. Newer majors exist (Vitest 4/5, MSW 3, TypeScript 6/7, Playwright 1.64, ESLint 10); the notes in lessons 02, 06, 09, 13 and 14 cover what changed. Tool configuration details change; the *principles* are what is assessed.

How the assessment is scored (summary)

Category	Parts	Points
Explanation	A (16 questions × 3)	48
Reasoning	B (8 × 3) + E (5 × 4)	44
Debugging	C (6 × 6)	36
Implementation	D (4 × 10)	40
Total		**168**

Pass: ≥ 80% overall (≥ 135 / 168) **and** ≥ 70% in *each* category (Explanation ≥ 34, Reasoning ≥ 31, Debugging ≥ 26, Implementation ≥ 28). Full rubric and promotion criteria are at the end.

Part A — Conceptual explanation (48 points)

Answer each in 4–8 sentences (or a short list). Each question is worth 3 points: one point per key idea in the answer key.

A1. (Lesson 01) Describe the test pyramid, the testing trophy, and the honeycomb as models. What does each optimise for, and what would make you choose a different shape for a given product? Where does static analysis fit?

A2. (Lessons 01, 16) Define "testing behaviour rather than implementation". Give the *refactor test* and explain the two failure directions of a bad test (over- and under-sensitivity), including how mutation testing relates.

A3. (Lesson 02) Distinguish test runner, assertion library, and test environment. Why does a green Vitest run not imply the TypeScript compiles? What does module-mock hoisting (`vi.mock`) mean for how you write the factory?

A4. (Lesson 03) State the Testing Library query priority and why it is ordered that way. Explain `getBy` / `queryBy` / `findBy` and why `user-event` is preferred over `fireEvent`.

A5. (Lesson 04) Explain what `act` does for React tests, how `waitFor` / `findBy` poll, and what an "not wrapped in act(...)" warning is really telling you. Why is a fixed sleep wrong?

A6. (Lesson 05) When do you test a custom hook with `renderHook` and when through a component? Name two hook-specific contract properties worth asserting (beyond returned values).

A7. (Lesson 06) Compare mocking `fetch` directly, mocking your own `api` module, and using MSW. What is mock drift and what are three defences? What does `onUnhandledRequest: 'error'` buy you?

A8. (Lesson 07) For a component that loads data, list the interaction states that deserve tests, and say what you assert about the *request* itself (as opposed to the UI). How do you test that a stale response from an earlier request does not overwrite a newer one?

A9. (Lesson 08) What distinguishes an integration test from a unit test in a React app? Which collaborators do you keep real and which do you replace, and why? Why must each test create fresh providers (router, query client, store)?

A10. (Lesson 09) Explain Playwright's auto-waiting and web-first assertions, browser contexts as an isolation mechanism, and how traces help debug CI failures. What belongs in E2E and what does not?

A11. (Lesson 10) What can automated accessibility tools detect and not detect? Why do role-based queries count as accessibility assertions? Why must colour-contrast checks run in a real browser rather than jsdom?

A12. (Lesson 11) Define a component contract and list at least five of its dimensions. Explain a shared contract suite and the controlled/uncontrolled contract. How do typed fixtures and schema validation reduce mock drift?

A13. (Lesson 12) List five sources of non-determinism in screenshots and a control for each. Distinguish per-pixel threshold from pixel budget and explain why baselines must be generated in the same environment that verifies them.

A14. (Lesson 13) What does TypeScript prove and what can it not prove? Explain type erasure, why `JSON.parse(x) as T` is not validation, and what `noUncheckedIndexedAccess` and exhaustiveness checking with `never` provide.

A15. (Lesson 14) Explain why `react-hooks/rules-of-hooks` is a correctness rule, give the escalation ladder for responding to an `exhaustive-deps` warning, and describe a ratchet strategy for adopting rules in a legacy codebase.

A16. (Lesson 15) Explain fail-fast pipeline ordering, the aggregator-job pattern and why it exists, why retries must be paired with flake reporting, and what is unsafe about `pull_request_target` combined with checking out PR code.

Part B — Code prediction (24 points; 3 each)

Assume React 19, Vitest with jsdom, `@testing-library/react`, `@testing-library/jest-dom` matchers registered, `user-event` v14, StrictMode **off** unless stated. State the outcome and justify it in 1–3 sentences.

B1.

```
function Late() {
  const [v, setV] = useState<string | null>(null);
  useEffect(() => {
    const id = setTimeout(() => setV('Ready'), 50);
    return () => clearTimeout(id);
  }, []);
  return <p>{v ?? 'Loading'}</p>;
}

test('late', async () => {
  render(<Late />);
  expect(screen.queryByText('Ready')).toBeNull();
  expect(await screen.findByText('Ready')).toBeInTheDocument();
});
```

Does the test pass? Is any `act` warning expected? Why?

B2.

```
const onChange = vi.fn();
render(<input aria-label="Name" value="a" onChange={onChange} />);
await userEvent.setup().type(screen.getByLabelText('Name'), 'bc');
```

What is the input's final value? How many times is `onChange` called, and what does each call observe as `event.target.value`?

B3.

```
const onClick = vi.fn();
render(<button disabled onClick={onClick}>Save</button>);
await userEvent.setup().click(screen.getByRole('button', { name: 'Save' }));
```

Does `getByRole` find the button? Is `onClick` called?

B4.

```
vi.useFakeTimers();
const user = userEvent.setup();
render(<button>OK</button>);
await user.click(screen.getByRole('button'));
```

What happens, and why? What is the minimal fix?

B5.

```
const track = vi.fn();
function Ping() {
  useEffect(() => { track('view'); }, []);
  return null;
}
render(<StrictMode><Ping /></StrictMode>);
expect(track).toHaveBeenCalledTimes(1);
```

Does it pass in the Vitest/jsdom environment with a development build of React? Explain, and say what you would assert instead if the requirement is "one analytics event per view".

B6.

```
type S = { k: 'a'; a: number } | { k: 'b'; b: string };
function f(s: S) {
  switch (s.k) {
    case 'a': return s.a;
    case 'b': return s.b;
  }
}
```

What is the inferred return type? Now a third member `{ k: 'c' }` is added to `S` and `f` is not updated. With only `"strict": true`, what does the compiler report for `f`, and what two additions would make the omission a compile error?

B7.

```
const text = await page.locator('#status').textContent();
expect(text).toBe('Done');
// versus
await expect(page.locator('#status')).toHaveText('Done');
```

Which of the two retries until the condition holds (within timeout)? Which is likely to be flaky for content that appears asynchronously, and why?

B8. A visual test uses `maxDiffPixelRatio: 0.02` on a 1000 × 500 px screenshot. How many differing pixels are tolerated? A teammate says "a missing 100×100 icon would still pass". Is that correct?

Part C — Debugging challenges (36 points; 6 each)

For each: name the **root cause** (2 points), give the **fix** with code or precise description (2 points), and describe how you would **verify** the fix and prevent recurrence (2 points).

C1. Order-dependent failure.

```
// test-utils.tsx
const queryClient = new QueryClient();
export const renderWithClient = (ui: ReactElement) =>
  render(<QueryClientProvider client={queryClient}>{ui}</QueryClientProvider>);

// Users.test.tsx
test('A: lists users', async () => {
  server.use(http.get('/api/users', () => HttpResponse.json([{ id: '1', name: 'Ada' }])));
  renderWithClient(<Users />);
  expect(await screen.findByText('Ada')).toBeInTheDocument();
});
test('B: shows an error on 500', async () => {
  server.use(http.get('/api/users', () => new HttpResponse(null, { status: 500 })));
  renderWithClient(<Users />);
  expect(await screen.findByRole('alert')).toBeInTheDocument();
});
```

Test B passes alone but fails (times out) when run after A.

C2. The `act` warning.

```
function Saver() {
  const [msg, setMsg] = useState('');
  return (
    <>
      <button onClick={() => setTimeout(() => setMsg('Saved'), 100)}>Save</button>
      <p>{msg}</p>
    </>
  );
}
test('saves', async () => {
  const user = userEvent.setup();
  render(<Saver />);
  await user.click(screen.getByRole('button', { name: 'Save' }));
  await new Promise((r) => setTimeout(r, 200));
  expect(screen.getByText('Saved')).toBeInTheDocument();
});
```

The test passes but logs "An update to Saver inside a test was not wrapped in act(...)". The team proposes silencing `console.error` in setup.

C3. Date test that passes in CI but fails on laptops in the US.

```
function Joined({ iso }: { iso: string }) {
  return <p>Joined {new Date(iso).toLocaleDateString('en-US')}</p>;
}
test('formats date', () => {
  render(<Joined iso="2026-03-01T00:30:00Z" />);
  expect(screen.getByText('Joined 3/1/2026')).toBeInTheDocument();
});
```

C4. Green tests, inaccessible control.

```
export function CloseButton({ onClose }: { onClose: () => void }) {
  return <button data-testid="close" onClick={onClose}><XIcon /></button>;
}
test('closes', async () => {
  const onClose = vi.fn();
  render(<CloseButton onClose={onClose} />);
  await userEvent.setup().click(screen.getByTestId('close'));
  expect(onClose).toHaveBeenCalled();
});
```

An axe scan in Playwright reports a violation on this control. Why did the unit test not catch it, and how do you fix both the component and the test?

C5. Stale closure caught by lint but suppressed.

```
function Ticker() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setCount(count + 1), 1000);
    return () => clearInterval(id);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);
  return <p>{count}</p>;
}
```

After 3 seconds the UI shows `1`. Explain, fix it without removing the "run once" intent, and write the fake-timer test that would have caught it.

C6. Flaky E2E.

```
test('save shows a toast', async ({ page }) => {
  await page.goto('/settings');
  await page.click('#save');
  await page.waitForTimeout(1000);
  expect(await page.locator('.toast').count()).toBe(1);
});
```

It fails ~8% of the time on CI. Diagnose and rewrite.

Part D — Implementation exercises (40 points; 10 each)

Write real code. Each is graded on correctness, adherence to the principles taught (behaviour-level, accessible queries, no sleeps, isolation), and clarity.

D1. Integration test with MSW (10). Component `UserSearch`: a text input labelled "Search users", a button "Search"; on submit it fetches `GET /api/users?q=<query>`; while loading it shows `role="status"` text "Searching…"; on success it renders results in a `<ul aria-label="Results">` of `<li>` names, or the text "No users found" when empty; on HTTP 500 it shows `role="alert"` text "Could not search users". Write three tests (success, empty, error) using MSW 2.x handlers, `user-event`, a fresh `QueryClient`-or-equivalent per test (you may assume a `renderWithProviders` helper exists), no sleeps, no snapshots, and assert that the request carried the query string typed by the user.

D2. Accessibility and keyboard test (10). Given:

```tsx
export function Disclosure({ title, children }: { title: string; children: ReactNode }) {
  const id = useId();
  const [open, setOpen] = useState(false);
  return (
    <div>
      <button aria-expanded={open} aria-controls={id} onClick={() => setOpen((o) => !o)}>
{title}</button>
      <div id={id} hidden={!open}>{children}</div>
    </div>
  );
}
```

Write tests covering: role/name, initial collapsed state (`aria-expanded` and content not visible), keyboard activation by Enter and by Space, `aria-controls` pointing at the region, and an axe check in the expanded state (state which package/matcher setup you assume and flag it for verification).

D3. Type-level correctness (10). Model a checkout async state as a discriminated union (`idle | submitting | success{orderId} | error{message, retryable}`), write a reducer with actions `SUBMIT`, `SUCCEEDED(orderId)`, `FAILED(message, retryable)`, `RESET` that only permits valid transitions, enforce exhaustiveness with `never`, write a component fragment that renders each state, and add at least three negative type tests using `@ts-expect-error`. State which compiler flags and which CI command make those negative tests meaningful.

D4. CI workflow (10). Write a GitHub Actions workflow (YAML) with: triggers for pull requests and `merge_group`; least-privilege permissions; concurrency that cancels superseded PR runs; parallel jobs for static checks (format, lint with zero warnings, typecheck), unit tests sharded three ways, and a production build; an E2E job that depends on the build and is sharded four ways using Playwright with artifact upload on failure; and a single aggregator job suitable as the only required branch-protection check. Flag any action versions or CLI flags you would verify.

Part E — Architecture and reasoning (20 points; 4 each)

Answer in a structured way: criteria, options considered, recommendation, trade-offs, what you would measure. About 150–250 words each.

E1. You are adding a new multi-step checkout (address, shipping method, payment via a third-party hosted field, confirmation). Allocate tests across layers (static, unit, component/integration, a11y, visual, E2E), name what runs on every PR versus nightly, and say what you deliberately will not test and why.

E2. A five-year-old React app has 2,000 DOM snapshot tests (most updated reflexively), a 35-minute CI with ~6% flaky E2E runs, loose TypeScript (`strict: false`, many `any`), and no accessibility checks. You have one team of five for one quarter. Produce a prioritised 90-day plan with success metrics.

E3. You own a design system consumed by eight product teams. Decide how to divide responsibility among contract tests, accessibility tests, visual regression, type tests, and consumer smoke tests, and how baseline/contract changes are reviewed and versioned.

E4. The backend team ships API changes without notice about once a month and your UI tests all pass against hand-written fixtures. Compare options (OpenAPI-generated types, runtime schemas at the boundary, consumer-driven contract testing, E2E against staging, recorded fixtures) and recommend a combination, with costs.

E5. Leadership asks you to "require 90% coverage on every PR" and "re-run failed tests automatically up to five times". Respond with a reasoned proposal that achieves the intent (quality and green builds) without the failure modes of those two policies.

Answer key

> Attempt the assessment first. Each answer lists the **key ideas**; award one point per key idea in Part A (max 3), and use the stated splits elsewhere. Accept any equivalent correct formulation. Where tool details are version-sensitive, accept "verify in docs" as a correct disposition.

Part A — Answers

A1. Key ideas:

1. **Pyramid**: many fast unit tests, fewer integration, fewest E2E; optimises feedback speed and cost; assumes logic lives in testable units and UI tests are expensive/flaky.
2. **Trophy**: static analysis at the base, then unit, with the *largest* layer being integration tests (component + providers + network boundary), few E2E; optimises confidence per cost for UI-heavy apps where bugs live in wiring (Testing Library's philosophy: tests resembling usage). **Honeycomb**: emphasises integration tests among services (microservice contexts), minimal unit/E2E.
3. Shape selection depends on where defects occur and test cost: algorithm-heavy domain (pricing engine) → more unit tests; UI glue-heavy app → trophy; payment/regulatory journeys → add more E2E; large micro-frontend/service ecosystems → contract and integration emphasis. Static analysis (types, lint) is the cheapest layer, catching a class of bugs before any test runs. *Common wrong answers:* "the pyramid says always 70/20/10" (ratios are illustrative); "E2E is bad" (it's expensive, not bad).

A2. Key ideas:

1. Behaviour = what a user or public consumer can observe (rendered semantics, callbacks, network requests, URL); implementation = state names, hooks, child structure, class names, call counts incidental to the design.
2. **Refactor test**: would this test fail if I refactor internals without changing behaviour (over-sensitivity → brittle)? Would it fail if I broke the behaviour it claims to verify (under-sensitivity → false confidence)? Plus: is the failure message diagnostic?

3. **Mutation testing** automates the second question: it introduces small code changes (mutants) and checks whether tests fail; surviving mutants reveal weak assertions. It's slow, so apply selectively to critical modules; "equivalent mutants" cause noise. *Edge:* counts are behaviour when they are the requirement (debounce, idempotent submit).

A3. Key ideas:

1. **Runner** discovers/executes tests and reports (Vitest); **assertion library** provides `expect` and matchers (Vitest's `expect` plus jest-dom extensions); **environment** supplies globals (jsdom/happy-dom simulating a browser, or Node), with no layout engine.
2. Vitest/Vite transpile TypeScript by stripping types (esbuild/SWC); no whole-program type check occurs, so `tsc --noEmit` must be a separate CI step.
3. `vi.mock` calls are **hoisted** above imports so mocked modules are in place before the module under test imports them; the factory therefore cannot reference variables declared later in the file unless created with `vi.hoisted`; use `importOriginal` for partial mocks. *(Accept "verify details in Vitest docs".)*

A4. Key ideas:

1. Priority: `ByRole` → `ByLabelText` → `ByPlaceholderText` → `ByText` → `ByDisplayValue` → `ByAltText` → `ByTitle` → `ByTestId`; ordered from how everyone (incl. assistive technology) perceives the page to implementation-only hooks; role queries also assert accessibility semantics.
2. `getBy` throws if absent/multiple (synchronous, must exist now); `queryBy` returns null (for absence assertions; throws on multiple); `findBy` returns a promise and waits (for async appearance).
3. `user-event` simulates the real event sequence (pointer events, focus, keyboard events, input events) and respects disabled/readonly states; `fireEvent` dispatches a single synthetic event bypassing these sequences, so it can pass on controls real users cannot operate. `user-event` v14 API is async and must be awaited.

A5. Key ideas:

1. `act` ensures React processes and flushes all updates/effects triggered by an action before you assert, emulating the browser's update cycle; RTL wraps `render`, `user-event` interactions (via its async wrapper), `findBy`, `waitFor` in `act`/environment flags so you rarely call it directly.
2. `waitFor` re-runs a callback on an interval and on DOM mutations until it stops throwing or times out (default roughly one second; configurable); `findBy` = `getBy` + `waitFor`.
3. The warning means React state updated outside an act-aware, awaited flow: something resolved later than the test knew about (timer, promise, subscription). The remedy is to await the observable outcome, flush the timers/promises inside `act`, or cancel/clean up the source — never silence. Sleeps are wrong because they encode a guess about timing: slow CI fails, fast CI wastes time, and they don't establish that the condition holds.

A6. Key ideas:

1. `renderHook` (with a `wrapper` for providers) is appropriate when the hook is a reusable unit with a rich contract independent of any UI (inputs/outputs, `rerender` with new props, `unmount`); testing through a component is better when the hook is only meaningful in one component, or when DOM behaviour (focus, events) is the point; avoid testing private hooks separately from the behaviour they enable.

2. Hook-specific contract properties: **referential stability** of returned callbacks/objects across re-renders (when documented), **cleanup** on unmount and on dependency change (subscriptions, timers, abort), behaviour under `rerender` with changed props, StrictMode double-effect safety, errors thrown when used outside a required provider.
3. Updates triggered outside RTL's wrappers need `act`; `result.current` is read fresh after each act (don't destructure early). *(Accept any two properties.)*

A7. Key ideas:

1. **Mocking** `fetch` couples tests to the call form and ignores real HTTP semantics; **mocking your** `api` **module** couples to file/function names and removes URL/headers/parsing from test coverage; **MSW** intercepts at the network boundary so the real client code runs and survives swapping fetch/Axios/TanStack Query or refactoring modules.
2. **Mock drift**: fixtures diverge from the real API so tests pass while production fails. Defences: typed fixtures/handlers generated from a spec; runtime schema validation at the boundary reused in fixtures; consumer-driven/provider-verified contract tests; E2E against a real environment; periodically recorded real responses.
3. `onUnhandledRequest: 'error'` makes an unexpected request fail the test loudly instead of silently hitting the network or being ignored, revealing missing handlers and accidental requests. Per-test overrides use `server.use`, reset in `afterEach` via `resetHandlers`.

A8. Key ideas:

1. States: initial/idle, loading (indicator and a11y status), success with data, empty, error (including partial errors), retry after error, refetch/background refresh states, pagination/boundaries, offline/timeouts if supported, and permission (401/403) handling.
2. Request assertions: assert on what is sent (method, URL, query params, headers like auth, body) at the MSW handler boundary or by capturing the request, and that the right number of *distinct* requests are made only when counts are the requirement (e.g. dedupe, debounce); not on internal function calls.
3. Stale responses: control the ordering using deferred promises or handlers that respond with different delays/resolvers; start request 1, then request 2, resolve 2 then 1, and assert the UI shows result 2 (requires cancellation via `AbortController` or ignoring outdated responses/query-key based cache). (Phase 6 lesson 02.)

A9. Key ideas:

1. Integration test renders a meaningful slice (page/feature with its real child components, state, routing, and data layer) and drives it as a user; unit test isolates a single function/component with collaborators replaced. Integration tests catch wiring bugs units cannot; units diagnose logic precisely.
2. Keep real: your components, hooks, state management, router, form libraries, validation, query client. Replace: network (MSW), time, randomness, browser APIs missing in jsdom, third-party services/iframes. Reason: mocking your own code removes the behaviour under test and creates drift.
3. Fresh providers per test prevent shared state (caches, stores, history) leaking between tests which causes order dependence; e.g., a module-scope `QueryClient` returns cached data to later tests; set `retry: false` and short `gcTime` in test clients *(verify option names for your version)*.

A10. Key ideas:

1. **Auto-waiting**: actions wait for the element to be attached, visible, stable, enabled, receiving events before acting; **web-first assertions** (`await expect(locator).toHaveText(...)`) retry until satisfied or timeout, unlike reading a value once.
2. **Browser contexts** are isolated sessions (cookies/storage) created cheaply per test; give isolation and parallelism; reuse authentication state via storage state to avoid logging in every test. **Traces** record actions, DOM snapshots, network, console, so failures can be inspected post hoc (`trace: 'on-first-retry'`).
3. E2E belongs to a few critical, cross-system journeys (login, checkout) running against a built app; not exhaustive permutations (unit/integration), not pixel-level checks everywhere (visual targeted), not logic that can be verified lower in the pyramid. Flakiness avoided with locators by role, no sleeps, hermetic data.

A11. Key ideas:

1. Automation detects structural/syntactic issues (missing names/labels/alt attributes, invalid ARIA, duplicate ids, some contrast failures in real browsers) but cannot judge meaning (alt text quality, logical order), full keyboard operability, focus management quality, or screen-reader experience; typically only a portion of issues overall (vendor claims vary: verify before quoting); a clean axe run is not conformance. `incomplete` results require human review.
2. Role+name queries use the same accessibility-tree information assistive tech uses; if a control cannot be found by role and name, a screen reader user likely cannot identify it either; test ids bypass this.
3. jsdom has no layout/paint engine, so computed colours, geometry, overlays, and visibility-by-layout are unavailable or unreliable; contrast needs rendered colours and backgrounds; use Playwright/Storybook in a real browser.

A12. Key ideas:

1. **Contract**: behaviours consumers can rely on, separate from implementation. Dimensions (any five): props/types, events/callback payload semantics, rendered semantics (role/name/state), interaction states, controlled/uncontrolled, composition rules and misuse errors, pass-through (`ref`, `className`, rest props), keyboard/focus/a11y, type-level guarantees, non-functional commitments.
2. **Shared contract suite**: a function taking an implementation and registering tests so that old/new/wrapper implementations run the same suite (migration safety); controlled contract: when `value` is provided the component displays it regardless of its own events, calling `onChange` with the proposed new value; uncontrolled: `defaultValue` seeds internal state.
3. Typed fixtures from the spec plus schema parsing in builders (fixtures cannot be invalid), the same schemas used by production fetch code, so spec changes fail compile/parse in tests; plus provider verification where services deploy independently.

A13. Key ideas:

1. Sources and controls (any five): fonts (pinned container, await `document.fonts.ready`, local fonts); animations (disable animations, reduced motion); time (freeze clock); data (fixtures/seeds); third-party/network content (stub/mask); platform rasterisation (same OS/container image); viewport/DPR (pin); scrollbars/caret/hover (hide/move mouse); lazy images (wait for load).

2. **Threshold** = per-pixel colour distance sensitivity; **pixel budget** = how many pixels (`maxDiffPixels`) or fraction (`maxDiffPixelRatio`) may differ; both loosen the test, so keep small and measured; mask dynamic regions instead.
3. Text rendering and fonts differ across OS/browser versions, so baselines from a laptop will not match CI; generate and verify in the same pinned image; baseline updates reviewed like code.

A14. Key ideas:

1. Proves (when sound): call sites match signatures, nullability is handled, union variants are handled exhaustively, refactors propagate, generic relationships hold. Cannot prove: runtime data matches declared types, business logic correctness, behaviour/timing/a11y, absence of `any` / `as` escapes, third-party data.
2. **Type erasure**: types don't exist at runtime; `as` is a compiler assertion with no runtime effect; `JSON.parse(x) as T` / `res.json()` returning `any` provide no validation; validate at boundaries with schemas or narrow from `unknown`.
3. `noUncheckedIndexedAccess` makes `arr[i]` possibly `undefined`; exhaustiveness via `never` (`assertNever`) makes adding a union member a compile error at all unhandled switches; `tsc --noEmit` as separate CI step; `@ts-expect-error` as negative type tests. *(Any two of these three tools with explanation earn the point.)*

A15. Key ideas:

1. React associates hook state with call order per component; conditional/looped hooks change the order between renders, mismatching state; so violations are bugs, not style.
2. Escalation ladder for `exhaustive-deps`: (a) do you need an Effect at all (derive in render, handle in event handler)? (b) move values/functions inside the effect, hoist constants, use functional updates, stabilise identities at the source; (c) if you need the latest value without re-synchronising, use the documented tool (`useEffectEvent` where available: verify status/version); (d) only then a line-level disable with a reason. Never file-wide or global disables.
3. Ratchet: baseline existing violations (bulk suppression or per-directory overrides), enforce on new/changed code, auto-fix mechanically in dedicated PRs, track burn-down, `--max-warnings 0` in CI, require reasons and report unused disable directives.

A16. Key ideas:

1. Order by cost and failure likelihood: format/lint/typecheck first (seconds–minutes), then unit/integration, then build, then E2E/visual; parallelise independent jobs; fail fast to avoid wasting slow work.
2. **Aggregator job** with `needs` and `if: always()` is the single required status check, so renaming jobs or path-filter skipping doesn't leave a required check pending forever or silently unrequired.
3. Retries (0–1) with classification as "flaky" and reporting/tracking plus owner and SLA/quarantine; blind retries hide nondeterminism and erode trust. `pull_request_target` runs with base-repo secrets/write token; checking out and running PR code there lets attackers exfiltrate secrets. Use `pull_request` (no secrets for forks) or isolated two-stage workflows.

Part B — Answers

B1. *Passes; no* `act` *warning.* `render` is synchronous; the timeout hasn't fired, so `queryByText('Ready')` is `null`. `findByText` polls inside RTL's async/`act` wrapper until the timer fires at ~50 ms and the update renders, well within the default timeout. Because the update occurs while an awaited RTL utility is active, React's `act` environment is satisfied. (Variation: removing the `await` on `findByText` would leave the update outside any awaited flow and may warn or fail the assertion.) *Scoring:* 1 pt outcome (passes, no warning), 1 pt `findBy` waits/polling, 1 pt act reasoning.

B2. *Final value* `'a'`*;* `onChange` *called twice.* Controlled input: React restores the DOM value to the prop after each event handler because state did not change. Typing `b` dispatches an input event whose `target.value` is `'ab'` (first call); React reverts to `'a'`; typing `c` yields `'ac'` (second call). Value never changes because the parent never updates `value`. Calls observe `'ab'` then `'ac'` (not `'abc'`). *Scoring:* 1 pt value, 1 pt call count 2, 1 pt observed values.

B3. `getByRole` *finds it;* `onClick` *is not called.* Disabled buttons remain in the accessibility tree (role `button`), and `getByRole` doesn't exclude disabled elements. `user-event` respects the `disabled` attribute and doesn't dispatch click events to disabled controls (a real browser behaves the same); `fireEvent.click` on a disabled button also doesn't trigger React's handler, but the key point is realism. *Scoring:* 1 pt finds, 1 pt not called, 1 pt reasoning.

B4. *The test hangs and times out* (default Vitest test timeout, 5 s unless configured; `testTimeout` default verified in the v3 docs). Two `setTimeout(0)` waits are involved: `user-event`'s inter-action delay (default `delay: 0`) and Testing Library's own `asyncWrapper` drain after each `user.*` call; with fake timers neither fires unless time is advanced. Minimal fix (verified on Vitest 3.2.7): `vi.useFakeTimers({ shouldAdvanceTime: true })`, or stub a global `jest` (`vi.stubGlobal('jest', { advanceTimersByTime: vi.advanceTimersByTime.bind(vi) })`) together with `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })`. `advanceTimers` or `delay: null` alone are *not* enough. Also restore real timers in `afterEach`. *Scoring:* 1 pt hang, 1 pt cause (internal timers), 1 pt fix.

B5. *Fails with 2 calls* when React runs in development mode (Vitest's default `NODE_ENV=test` loads React's development build, where StrictMode double-invokes mount effects: mount → cleanup → mount). Since the effect has no cleanup that undoes the tracking, the event fires twice. Production does not double-invoke. If the requirement is "one analytics event per view", the code should dedupe (e.g., key by view in a module-level `Set`/dedupe at the analytics client, or fire from the event/route layer rather than a mount effect), and the test should assert on the *outbound analytics request/payload at the boundary* with dedupe logic, rendered under StrictMode to prove it; do not divide counts by two. *Scoring:* 1 pt result (2 calls), 1 pt cause (StrictMode dev double effect), 1 pt better assertion/design.

B6. *Return type* `string | number` (switch exhaustive). After adding `{ k: 'c' }`, the inferred return becomes `string | number | undefined` silently (fall-through returns `undefined`); with only `"strict": true` there's no error (since `noImplicitReturns` is not part of `strict`). Making it an error: enable `noImplicitReturns` (reports `TS7030: Not all code paths return a value.`, verified on TypeScript 5.9.3 and 7.0.2 with `noImplicitReturns` added to `strict`; with `strict` alone `tsc` reports nothing), and/or add `default: return assertNever(s)` (where `assertNever(x: never): never`), or a typed return annotation that excludes `undefined`, e.g. `function f(s: S): string | number`. *Scoring:* 1 pt `string | number`, 1 pt silent `undefined` after change, 1 pt the two remedies.

B7. *The web-first assertion retries* (`expect(locator).toHaveText`), re-evaluating until it passes or times out. The first form reads `textContent()` **once** (with auto-waiting only for the element to exist/attach, not

for its text to change), then asserts synchronously, so if the text updates after the read, it fails: flaky for asynchronous content. *Scoring:* 1 pt which retries, 1 pt which is flaky, 1 pt reasoning.

B8. *10,000 pixels* (1000 × 500 = 500,000 × 0.02). A 100 × 100 icon is exactly 10,000 pixels; if every pixel differed it would be at the limit (passes if ≤ the budget, depending on whether the comparison is `<=`); in practice a missing icon changes fewer than 100% of its pixels (some background pixels identical), so it would likely pass. So the teammate is *right*: this tolerance is too loose; use ~0.001 or less (≈500 px) measured empirically, and mask truly dynamic areas. *Scoring:* 1 pt number, 1 pt agreement with caveat, 1 pt remedy.

Part C — Answers

C1. Order-dependent failure. *Root cause (2):* `queryClient` is module-scoped, so its cache persists across tests. In B, the query key already holds data from A, so the component renders cached data immediately (stale-while-revalidate) and the background refetch that fails is retried (default `retry: 3` with exponential backoff, several seconds in total) before the error state is reached; depending on how the component treats error-with-existing-data, `role="alert"` does not appear within the `findBy` timeout. *(Exact status/retry semantics: verify in the TanStack Query docs for your version.) Fix (2):* create a new client per render with test defaults:

```
export function renderWithClient(ui: ReactElement) {
  const client = new QueryClient({ defaultOptions: { queries: { retry: false, gcTime: 0 } }
}); // v5 option names verified (`gcTime`; `cacheTime` in v4)
  return render(<QueryClientProvider client={client}>{ui}</QueryClientProvider>);
}
```

Verify/prevent (2): run B alone and after A, plus a shuffled run (`--sequence.shuffle`, flag verified); add a lint/code-review rule against module-level singletons in test utils; reset handlers in `afterEach`; consider a test asserting two successive renders don't share cache.

C2. The `act` warning. *Root cause (2):* The state update happens in a timer callback that fires while the test is awaiting a raw 200 ms sleep, outside any RTL-wrapped, awaited flow; React is in an act-environment and sees an un-acted update. The sleep merely hid the problem by waiting long enough for the DOM to change. *Fix (2):* await the observable outcome: `expect(await screen.findByText('Saved')).toBeInTheDocument();` (wrapped in `act` by RTL), or use fake timers and advance inside `act` (`await act(async () => { vi.advanceTimersByTime(100); })`). Remove the sleep. Do not silence `console.error`. *Verify/prevent (2):* the warning disappears without mocking console; the test no longer depends on wall-clock time; add the `testing-library/await-async-utils` and `testing-library/no-await-sync-queries` lint rules (both fired in the lesson 14 verified config run); optionally fail tests on unexpected `console.error` (with allowlist) so such warnings can't accumulate.

C3. Timezone dependence. *Root cause (2):* `toLocaleDateString` uses the runtime's local time zone; `2026-03-01T00:30:00Z` is 28 Feb evening in US time zones (UTC−5…−8) but 1 Mar in UTC; CI runs in UTC, developers' laptops don't. *Fix (2):* decide the product behaviour. If dates are shown in UTC: `new Date(iso).toLocaleDateString('en-US', { timeZone: 'UTC' })`. If shown in the user's zone: make the zone injectable/explicit (`Intl.DateTimeFormat(locale, { timeZone })`) and pass a fixed zone in tests; also pin `TZ` for the test run (`TZ=UTC vitest` or config) as a safety net. *Verify/prevent (2):* run the test with

`TZ=America/Los_Angeles` and `TZ=Asia/Tokyo` and `TZ=UTC`; add a test around the day boundary; document time-zone policy; lint/restricted syntax against bare `toLocale*` calls in code that should be zone-explicit (optional).

C4. Green tests, inaccessible control. *Root cause (2):* The button has no accessible name (icon only, `aria-hidden` not set on the icon, no label). The test used a test id and so never consulted the accessibility tree; axe's `button-name` rule flags it. *Fix (2):* component: `<button type="button" aria-label="Close" onClick={onClose}><XIcon aria-hidden="true" focusable="false" /></button>` (or visually hidden text). Test: `await user.click(screen.getByRole('button', { name: 'Close' }));` — this now fails if the name disappears. Remove the `data-testid` unless required for another purpose. *Verify/prevent (2):* add the component-level axe check (flag the matcher setup as version-dependent) and eslint-plugin-jsx-a11y; add the page scan in Playwright; document that icon-only controls require a name, and encode that in the props type (union requiring `aria-label` or children).

C5. Stale closure. *Root cause (2):* The effect closes over `count` from the first render (`0`); with `[]` dependencies it never re-runs, so each tick calls `setCount(0 + 1)`: the value stays `1`. The disabled lint rule was reporting exactly this. *Fix (2):* use a functional update, removing the dependency on `count` so "run once" is legitimate:

```
useEffect(() => {
  const id = setInterval(() => setCount((c) => c + 1), 1000);
  return () => clearInterval(id);
}, []);
```

and delete the `eslint-disable`. *Verify/prevent (2):* test:

```
test('ticks every second', () => {
  vi.useFakeTimers();
  render(<Ticker />);
  act(() => { vi.advanceTimersByTime(3000); });
  expect(screen.getByText('3')).toBeInTheDocument();
  vi.useRealTimers();
});
```

(restore timers in `afterEach`); lint rule requiring a description on disables and `reportUnusedDisableDirectives`; ban file-wide disables. Also under StrictMode the cleanup prevents double intervals; consider adding that run.

C6. Flaky E2E. *Root cause (2):* A fixed 1-second sleep plus a one-time `count()` read; if the toast appears after 1 s on a slow CI runner (or has already disappeared after an auto-dismiss), the count is 0 (or the toast is queued/animating). `page.click('#save')` uses an id selector (coupled), and `.toast` is a class selector. *Fix (2):*

```
await page.goto('/settings');
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.getByRole('status')).toHaveText(/saved/i);   // web-first assertion; auto-retries
```

Use a status/alert role on the toast (an a11y requirement as well); if the toast auto-dismisses quickly, configure the test environment with a longer duration or disable auto-dismiss; for "exactly one toast" use `await expect(page.getByRole('status')).toHaveCount(1)`. *Verify/prevent (2):* run with `--repeat-each=30` under CPU throttling (`--repeat-each` is a documented Playwright flag); inspect trace on failure; add `forbidOnly`, retries ≤ 1 with flake reporting; lint rule banning `waitForTimeout` (restricted syntax).

Part D — Reference solutions and rubrics

D1. Reference solution.

```tsx
import { http, HttpResponse } from 'msw';
import { screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { expect, test } from 'vitest';
import { server } from '../test/server';
import { renderWithProviders } from '../test/utils';
import { UserSearch } from './UserSearch';

function usersHandler(respond: (q: string) => Response | Promise<Response>) {
  return http.get('/api/users', ({ request }) => respond(new
URL(request.url).searchParams.get('q') ?? ''));
}

test('shows matching users and sends the typed query', async () => {
  const seen: string[] = [];
  server.use(usersHandler((q) => { seen.push(q); return HttpResponse.json([{ id: '1', name:
'Ada Lovelace' }]); }));
  const user = userEvent.setup();
  renderWithProviders(<UserSearch />);

  await user.type(screen.getByLabelText('Search users'), 'ada');
  await user.click(screen.getByRole('button', { name: 'Search' }));

  expect(await screen.findByText('Ada Lovelace')).toBeInTheDocument();
  expect(screen.getByRole('list', { name: 'Results' })).toBeInTheDocument();
  expect(seen).toEqual(['ada']);                          // count is a requirement here: one
request with the typed query
});

test('shows an empty state', async () => {
  server.use(usersHandler(() => HttpResponse.json([])));
  const user = userEvent.setup();
  renderWithProviders(<UserSearch />);
  await user.type(screen.getByLabelText('Search users'), 'zzz');
  await user.click(screen.getByRole('button', { name: 'Search' }));
  expect(await screen.findByText('No users found')).toBeInTheDocument();
});

test('shows an error alert on 500 and a loading status while pending', async () => {
  server.use(usersHandler(() => new HttpResponse(null, { status: 500 })));
  const user = userEvent.setup();
  renderWithProviders(<UserSearch />);
  await user.type(screen.getByLabelText('Search users'), 'ada');
  await user.click(screen.getByRole('button', { name: 'Search' }));
  expect(await screen.findByRole('alert')).toHaveTextContent('Could not search users');
```

```
  expect(screen.queryByRole('status')).not.toBeInTheDocument();
});
```

Rubric (10): three tests covering success/empty/error (3); network mocked with MSW at the boundary, per-test `server.use`, no module/fetch mocks (2); role/label queries, `user-event` awaited, `findBy` for async (2); asserts the request's query string (1); no sleeps/snapshots/implementation coupling and isolation via per-test providers (1); sensible negative controls (loading status absent after error; presence checks) and mention of `onUnhandledRequest: 'error'` / reset handlers (1). Testing the "Searching…" status with a deferred-promise handler is a worthwhile extra; omitting it costs at most 1 point from the negative-controls item. Deduct 2 for `getByTestId` / `container.querySelector`, 3 for sleeps.

D2. Reference solution.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { axe } from 'vitest-axe';                    // verified: vitest-axe 0.1.0 exports
`axe`; matchers via `expect.extend(matchers)`
import * as axeMatchers from 'vitest-axe/matchers';  // verified: provides toHaveNoViolations
import { expect, test } from 'vitest';
import { Disclosure } from './Disclosure';

expect.extend(axeMatchers);

function setup() {
  const user = userEvent.setup();
  const utils = render(<Disclosure title="Shipping details"><p>Free over $50</p>
</Disclosure>);
  return { user, ...utils, button: screen.getByRole('button', { name: 'Shipping details' }) };
}

test('is collapsed initially and exposes state', () => {
  const { button } = setup();
  expect(button).toHaveAttribute('aria-expanded', 'false');
  expect(screen.getByText('Free over $50')).not.toBeVisible();
});

test.each([['{Enter}'], [' ']])('toggles with keyboard key %j', async (key) => {
  const { user, button } = setup();
  await user.tab();
  expect(button).toHaveFocus();
  await user.keyboard(key);
  expect(button).toHaveAttribute('aria-expanded', 'true');
  expect(screen.getByText('Free over $50')).toBeVisible();
  await user.keyboard(key);
  expect(button).toHaveAttribute('aria-expanded', 'false');
});

test('aria-controls references the region', async () => {
  const { user, button } = setup();
  await user.click(button);
  const regionId = button.getAttribute('aria-controls');
  expect(regionId).toBeTruthy();
  expect(document.getElementById(regionId!)).toContainElement(screen.getByText('Free over
$50'));
});

test('has no detectable axe violations when expanded', async () => {
  const { user, button, container } = setup();
  await user.click(button);
```

```
  expect(await axe(container)).toHaveNoViolations();
});
```

Rubric (10): role/name query via `getByRole` (1); initial state assertions `aria-expanded` and non-visible content (2); keyboard tests for **both** Enter and Space using `user.keyboard` after tabbing (3); `aria-controls` verified against the DOM id (1); axe in the expanded state with `await` and a correct, flagged setup (2); no `fireEvent`/test ids (1). Notes: `aria-controls` on a hidden region is fine; also accept noting that the pattern lacks `type="button"` (bonus: inside a form it would submit) and that axe can't evaluate meaning. A good answer flags `toBeVisible` reliance on jsdom's handling of the `hidden` attribute and says to verify.

D3. Reference solution.

```tsx
export type Checkout =
  | { status: 'idle' }
  | { status: 'submitting' }
  | { status: 'success'; orderId: string }
  | { status: 'error'; message: string; retryable: boolean };

export type Action =
  | { type: 'SUBMIT' }
  | { type: 'SUCCEEDED'; orderId: string }
  | { type: 'FAILED'; message: string; retryable: boolean }
  | { type: 'RESET' };

function assertNever(x: never): never {
  throw new Error(`Unhandled: ${JSON.stringify(x)}`);
}

export function reducer(state: Checkout, action: Action): Checkout {
  switch (state.status) {
    case 'idle':
      return action.type === 'SUBMIT' ? { status: 'submitting' } : state;
    case 'submitting':
      if (action.type === 'SUCCEEDED') return { status: 'success', orderId: action.orderId };
      if (action.type === 'FAILED') return { status: 'error', message: action.message,
retryable: action.retryable };
      return state;
    case 'error':
      if (action.type === 'SUBMIT' && state.retryable) return { status: 'submitting' };
      if (action.type === 'RESET') return { status: 'idle' };
      return state;
    case 'success':
      return action.type === 'RESET' ? { status: 'idle' } : state;
    default:
      return assertNever(state);
  }
}

export function CheckoutStatus({ state }: { state: Checkout }) {
  switch (state.status) {
    case 'idle': return null;
    case 'submitting': return <p role="status">Placing order…</p>;
    case 'success': return <p role="status">Order {state.orderId} confirmed</p>;
    case 'error': return <p role="alert">{state.message}</p>;
    default: return assertNever(state);
  }
}
```

```ts
// checkout.test-d.ts (checked by tsc --noEmit, or `vitest --typecheck`; both verified)
import type { Checkout, Action } from './checkout';

const ok: Checkout = { status: 'success', orderId: 'o1' };
// @ts-expect-error — success requires orderId
const bad1: Checkout = { status: 'success' };
// @ts-expect-error — submitting carries no orderId (excess property for the variant)
const bad2: Checkout = { status: 'submitting', orderId: 'o1' };
// @ts-expect-error — FAILED requires retryable
const bad3: Action = { type: 'FAILED', message: 'x' };
void [ok, bad1, bad2, bad3];
```

Flags/commands: `strict: true` (and `noUncheckedIndexedAccess` if arrays are used), the test file included in `tsconfig` `include`, and `tsc --noEmit` (or Vitest's typecheck mode) in CI, because ordinary Vitest runs strip types and never evaluate `@ts-expect-error`. *Rubric (10):* correct discriminated union (2); reducer with valid-only transitions incl. retry guard (3); exhaustiveness via `never` in both reducer and component (2); three meaningful negative tests, each failing for the stated reason (2); flags and CI command stated, mention of `@ts-expect-error` semantic (unused directive → error) (1). Deduct for `any`, casts, boolean flags, or tests not wired to the type checker. *Accepted variants:* the reducer may also switch on `action.type` first; the key is invalid transitions being unrepresentable or ignored.

D4. Reference solution.

```yaml
name: CI
on:
  pull_request:
  merge_group:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.event.pull_request.number || github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  static:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with: { node-version-file: .nvmrc, cache: npm }
      - run: npm ci
      - run: npm run format:check
      - run: npx eslint . --max-warnings 0 --cache
      - run: npx tsc --noEmit

  unit:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix: { shard: [1, 2, 3] }
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with: { node-version-file: .nvmrc, cache: npm }
      - run: npm ci
      - run: npx vitest run --shard=${{ matrix.shard }}/3

  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with: { node-version-file: .nvmrc, cache: npm }
      - run: npm ci
```

```yaml
      - run: npm run build
      - uses: actions/upload-artifact@v7
        with: { name: dist, path: dist, retention-days: 3 }

  e2e:
    needs: build
    runs-on: ubuntu-latest
    timeout-minutes: 25
    container: mcr.microsoft.com/playwright:v1.64.0-noble      # match installed version
(current Docker docs use -noble)
    strategy:
      fail-fast: false
      matrix: { shard: [1, 2, 3, 4] }
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with: { node-version-file: .nvmrc, cache: npm }
      - run: npm ci
      - uses: actions/download-artifact@v8
        with: { name: dist, path: dist }
      - run: npx playwright test --shard=${{ matrix.shard }}/4
        env: { CI: 'true' }
      - uses: actions/upload-artifact@v7
        if: ${{ !cancelled() }}
        with:
          name: playwright-report-${{ matrix.shard }}
          path: |
            playwright-report/
            test-results/
          retention-days: 7

  ci-ok:
    if: always()
    needs: [static, unit, build, e2e]
    runs-on: ubuntu-latest
    steps:
      - run: |
          if [ "${{ contains(needs.*.result, 'failure') }}" = "true" ] || [ "${{
contains(needs.*.result, 'cancelled') }}" = "true" ]; then exit 1; fi
```

Verify list: action major versions, Playwright image tag matching the installed version, `--shard` flags (Vitest/Playwright), whether the Playwright webServer config serves `dist` (via `vite preview`), container/permissions caveats, artifact action semantics. *Rubric (10):* triggers incl. `merge_group` and PR (1); `permissions` least privilege and concurrency with scoped cancel (2); parallel static/unit/build with `npm ci`, caching, timeouts (2); unit sharding 3-way and E2E 4-way with `needs: build` and `fail-fast: false` (2); artifacts uploaded on failure with `!cancelled()` /equivalent (1); correct aggregator (`always()`,

`needs`, fails on failure/cancelled, treats skipped as OK) (1); explicit verify notes (1). Deduct for `|| true`, `npm install`, unpinned `@main`, secrets exposure, or no timeouts.

Part E — Answers (evaluation criteria and example answers)

Award 4 points: **criteria identified (1)**, **options considered with trade-offs (1)**, **justified recommendation (1)**, **measurement/risks/limits acknowledged (1)**.

E1. Checkout test allocation. *Example answer.* **Static**: strict TS, lint with hooks/a11y rules, schema-validated API boundary (every PR). **Unit**: pure logic: price/tax/shipping calculators and the checkout reducer, table-driven including boundary rounding (every PR, ms). **Integration (component + MSW)**: each step as a user (address validation errors, shipping options update total, payment failure paths, double-submit protection, refresh/back navigation state), fake payment-field adapter at the seam (every PR). **Accessibility**: role/name assertions are inherent; keyboard-only completion test; error summary focus; axe in jsdom for form components and Playwright axe scans per step/state (error state included) in CI; manual screen-reader pass per release. **Visual**: element-level baselines for step layouts at mobile/desktop with fixtures (PR if stable, otherwise merge-queue/nightly). **E2E**: one or two journeys on a built app with a sandbox/stubbed payment provider; nightly extended cross-browser. **Contract/boundary**: typed fixtures + schema parse; provider verification if services deploy independently. **Not tested (deliberately)**: the third-party hosted field's internals, pixel layout of every state combination (risk-based), exhaustive tax rules through the UI (belongs to unit tests), and implementation details like reducer state shape through the UI. Measure: escaped defects, flake rate, PR duration; revisit allocation by incident data.

E2. 90-day plan for a legacy app. *Example answer.* **Weeks 1–2: baseline and guardrails.** Measure: CI duration breakdown, flake rate by test, snapshot count/size, `any` count, a11y scan violation fingerprints, bug-escape history. Add CI gate for typecheck and lint with ratchet (no new violations), forbid *new* DOM snapshots (lint), `forbidOnly`, flake reporting. **Weeks 3–6: stabilise signal.** Triage flaky E2E top offenders (sleeps → web-first assertions; shared state; seed data), quarantine with owners; parallelise/shard CI to get PR required path < ~15 min; run `strict` per-directory for new code, `noImplicitAny` first. **Weeks 6–10: replace value-less tests.** Delete snapshot tests with no bug-catching history; rewrite the highest-churn ones to behaviour tests with MSW; add a11y scan on top five journeys (serious/critical ratchet). **Weeks 10–13: sustain.** Dashboards, ownership, documentation, lint for tests; retrospective. *Metrics:* PR CI p90, flake rate (<1–2%), snapshot count down, new-code `any` = 0, a11y serious violations on key routes trending to 0, test-caused PR rework (refactor-induced failures). Risks: team capacity, hidden coupling when deleting snapshots (mitigate by replacing around critical flows first). Trade-off: not attempting 100% migration; prioritise by risk and change frequency.

E3. Design system responsibilities. *Example answer.* **Library**: per-component contract suites (props/events/pass-through/controlled-uncontrolled), keyboard/focus tests for complex widgets, axe across states in jsdom plus real-browser Storybook a11y/contrast, type tests for generic/polymorphic APIs, visual regression per story in a pinned environment (hosted service or Playwright against static Storybook), build/packaging smoke tests importing the published bundle in a minimal consumer app. **Consumers**: composition-level tests (labels supplied, flows), page-level a11y scans, no assertions on library DOM structure/classes. **Review/versioning**: semantic versioning with contract changes = breaking; visual baseline changes require design-system owner approval and PR descriptions with before/after; baselines changed by dependency/browser upgrades isolated in their own PR; codemods

and deprecation windows; changelog generated from changesets *(verify tool)*. Trade-offs: visual tests cost/flake vs value; per-story volume (use change detection); contract suites ossify APIs (balance by only pinning documented behaviours). Measure: consumer breakage incidents, a11y regressions, visual false-positive rate, review turnaround.

E4. API drift. *Example answer.* Criteria: drift frequency, team boundaries, available specs, cost, speed of feedback, false-confidence risk. Options: **generated types from OpenAPI/GraphQL** (cheap, compile-time catch if the spec is accurate and fetched in CI; useless if spec lags); **runtime schemas at the boundary** reused by fixtures (catches real payload drift at runtime in dev/staging and in tests; cost: CPU/bundle; handle failures with logging and graceful degradation); **consumer-driven contracts (Pact-style)** (strong cross-team protection; highest process/infra cost, valuable when independent deploys and no spec governance); **E2E against staging** (realistic but slow/flaky; a few smoke tests); **recorded fixtures** (cheap realism; stale unless refreshed). Recommendation: spec-generated types + boundary schemas shared with MSW builders as the default; a nightly job replaying a handful of requests against staging with the schemas to detect drift; add provider verification only if incidents continue or no spec exists. Also add production monitoring of schema parse failures (without PII). Measure: drift incidents per quarter, time to detect, false-positive rate.

E5. Coverage and retry policies. *Example answer.* Intent: prevent regressions and keep builds trustworthy. Problems: a global coverage number is gameable and measures execution not assertion; auto-retrying 5× hides flakiness and multiplies CI time while eroding trust. Proposal: (1) **Coverage as ratchet**: set thresholds at current values, never allow decreases without justification; stricter per-path floors for critical modules (pricing, permissions); report diff coverage on changed lines as a review aid, not a hard gate; complement with mutation testing on critical modules quarterly. (2) **Retries ≤ 1** in CI with Playwright/Vitest classifying "flaky" and failing the build if flaky count exceeds a budget; auto-create tickets with trace artifacts; owners and SLA; quarantine job after SLA; dashboards on flake rate. (3) Add leading indicators: PR CI p90, change-failure rate, escaped-defect rate. (4) Explain the cost of the proposed policies using data from our own pipeline (e.g. average minutes burned by 5× retries). Trade-offs: slower initial gate adoption; some teams will feel constrained by per-path floors; mitigate with documented exceptions. Measure whether escaped defects decrease and green-build trust (re-run rate) improves.

Scoring rubric

Part	Item points	How to award
A (Explanation, 48)	3 each	1 point per correct key idea (maximum 3). Half credit is not given within a key idea; award the point only if the idea is correct and stated (not merely a keyword). Penalise a confidently wrong claim by withholding one point.
B (Reasoning, 24)	3 each	Split in the key (typically outcome / mechanism / consequence or fix). A correct outcome with wrong reasoning earns at most 1 point.
C (Debugging, 36)	6 each	Root cause (2) — must name the *mechanism*, not the symptom; Fix (2) — correct and minimal, not suppression; Verify/prevent (2) — a concrete check plus a preventive measure.

Part	Item points	How to award
D (Implementation, 40)	10 each	Use the per-exercise rubric. Code that would not run is graded on intent but capped at 7/10. Any use of sleeps, snapshots as the main assertion, or implementation coupling in D1/D2 caps the item at 6/10.
E (Reasoning, 20)	4 each	Criteria (1), options/trade-offs (1), justified recommendation (1), measurement/risk (1). A generic answer with no trade-offs earns at most 2.

Category totals and thresholds

Category	Max	Pass (≥ 70%)	Pass overall target (80%)
Explanation (A)	48	34	—
Reasoning (B + E)	44	31	—
Debugging (C)	36	26	—
Implementation (D)	40	28	—
Overall	**168**	—	**135**

Self-assessment guidance

- **Honesty rule:** mark an item as correct only if your answer would have convinced a sceptical reviewer without the key's wording.
- Count "verify against current docs" answers as full credit where the key flags the detail as volatile.
- If you used notes or AI assistance for Parts A–C or E, the result is a *study aid score*, not a promotion score.

Promotion criteria

You may proceed to **Phase 9 (Advanced rendering and ecosystem)** when **all** of the following are true:

1. **Scores:** overall ≥ 135/168 and each category above its threshold (Explanation ≥ 34, Reasoning ≥ 31, Debugging ≥ 26, Implementation ≥ 28).
2. **Evidence of practice (not only paper):** you have, in a real repository (a Phase project or your own), (a) a Vitest + RTL + MSW integration test suite with per-test providers and no sleeps, (b) at least one keyboard/a11y test and one axe scan, (c) a type-level negative test run in CI via `tsc --noEmit`, (d) a flat ESLint config with hooks and test rules passing with `--max-warnings 0`, (e) a CI workflow with a gate job, and (f) at least one Playwright test using web-first assertions and a trace-on-retry configuration.
3. **Critical misconceptions resolved:** you can explain without notes why (i) a green Vitest run does not imply type correctness, (ii) `getByTestId` is a last resort, (iii) retries without flake reporting are harmful, (iv) coverage is not a quality target, (v) visual baselines must be generated in the verifying environment, and (vi) `as` and `any` void type guarantees.

4. **Calibration:** no single Part below 50% (a catastrophic gap in one part indicates a missing mental model even if the aggregate passes).

If you did not pass

Weak category / part	Revisit	Then do
Explanation, A1–A3	01, 02, 16	Re-derive answers aloud; summarise each lesson's §2 in your own words
A4–A6, B1–B4, C1–C2, D1	03, 04, 05	Redo lesson exercises C–D; write 5 tests for an existing component
A7–A9, D1, E4	06, 07, 08	Build an MSW-backed integration suite for a feature with loading/empty/error states
A10, B7, C6, D4	09, 15	Convert two sleeps-based E2E tests to web-first assertions; write a sharded workflow
A11, C4, D2	10	Run an axe scan + keyboard test on three components; fix findings
A12–A13, B8, E3	11, 12	Write a shared contract suite; set up one deterministic visual test in Docker
A14–A15, B6, C5, D3	13, 14	Add strict flags and `no-floating-promises`; model one async flow as a union with exhaustiveness
A16, E2, E5	15, 16	Write a flake policy and measure your own pipeline

Retake after at least one week with fresh attempts at the failed parts (the questions are intended to be re-derivable; memorising the key defeats the purpose).

Knowledge checkpoint (phase)

You should now be able to

- Choose test types by risk, cost and feedback speed; write behaviour-level tests that survive refactors.
- Control time, network, randomness and global state; test async, hooks, APIs, integration flows, and E2E journeys.
- Build accessibility, contract, visual, type-level and static-analysis safeguards, and assemble them into a trustworthy CI pipeline.
- Diagnose flaky, order-dependent, environment-specific and falsely-green tests.

Next: Phase 9 — Advanced rendering and ecosystem (see ROADMAP). Phase 8 tooling carries forward: server rendering, streaming and framework features in Phase 9 and production engineering in Phase 10 are tested with the same boundary-mocking, accessibility and CI disciplines (framework-specific test tooling: verify against current docs).